

TABLE OF CONTENTS

PRELIMINARIES	Page
Title Page	i
Approval Sheet	ii
Dedication	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi
List of Tables	vii
List of Figures	viii
Chapter 1 - INTRODUCTION	1
Background of the Study	2
Objective of the Study	5
Scope and Limitation of the Study	6
Chapter 2 - CONCEPTUAL FRAMEWORK	7
Review of Related Literature	7
Conceptual Model of the Study	10
Operational Definition of Terms	13
Chapter 3 - METHODOLOGY	15
Project Design	15
Project Development	19
Operation and Testing Procedure	23
Evaluation Procedure	28

Chapter 1

INTRODUCTION

Background of the Study

The Philippines exists in a state of constant geographical vigilance due to its placement along the "Ring of Fire," a reality that periodically subjects its citizens to significant seismic activity. Data provided by the Philippine Institute of Volcanology and Seismology (PHIVOLCS, 2023) highlights that the nation remains one of the most earthquake-prone regions on Earth, which transforms disaster preparedness from a routine task into a vital mission of communal survival. Within the bustling high-density business districts, the success of an earthquake drill is measured not just by compliance, but by the preservation of human life through seamless coordination. Large-scale exercises, such as the National Simultaneous Earthquake Drill (NSED), serve as critical opportunities to refine the habits of safety and collective resilience. As Punzalan and Mendoza (2018) observe in their analysis of disaster risk reduction, effective communication functions as the essential link protecting individuals and facilitating a swifter return to normalcy after a catastrophe.

The Digital Service Operations Team currently relies on a set of traditional earthquake protocols that center on building-wide public address systems and the manual tracking of personnel. In the heart of a genuine emergency, the speed at which safety instructions reach an employee whether through a desk phone or an internal digital network—is the determining factor between an orderly evacuation and a chaotic, dangerous rush for the exits. During these high-stakes moments, the psychological

comfort of being informed is a primary human need; every member of the workforce deserves to feel personally guided and accounted for in real-time (Lim & Rodrigues, 2018). However, as modern work environments expand in both size and complexity, the traditional reliance on human-operated directory menus or manual call lists creates significant bottlenecks that can delay life-saving actions.

The fundamental challenge lies in the inherent limitations of manual notification workflows, which are deeply vulnerable to the effects of human stress and physical fatigue. Historically, organizations have leaned on methods like human operators or manual directory menus to disseminate information, yet these methods often struggle under the weight of an actual disaster (Cutter & Emrich, 2016). Research comparing staff recall methods has found that while manual trees are familiar, they suffer from high variability in response times during unannounced drills (Homier et al., 2018). In the high-pressure atmosphere following a seismic event, even the most trained operators are susceptible to errors that may lead to fragmented messaging or missed check-ins (Kwok & Chan, 2012). These manual approaches lack the necessary agility to support a modern, distributed workforce, often leaving those on isolated floors or remote shifts feeling disconnected from the safety loop.

Evidence suggests that current notification strategies are increasingly unable to bridge the gap between centralized office staff and team members working in diverse or remote locations. Alberts and Hayes (2008) point out that older communication infrastructures often fail to scale effectively when they are needed most, leaving vulnerable gaps in a company's safety net. Manual headcounts and physical check-in procedures are notoriously slow and are frequently documented as being prone to

inaccuracies that misdirect the attention of emergency responders (Lim & Rodrigues, 2018). Furthermore, recent studies by non-governmental organizations in the Philippines indicate that many local institutions still lack the automated technology required for real-time hazard alerts, leaving them to rely on word-of-mouth or isolated decisions (People in Need, 2022). At Digital Service Operations (DSO), the existing reliance on localized announcements means that critical safety directives might not reach every team member simultaneously, resulting in inconsistent response times.

This communication deficit is often rooted in a reliance on "one-size-fits-all" broadcast alerts and manual triggers, which frequently lead to a paralyzing influx of information for the end-user. When safety messages lack a personal context or location-specific details, employees may inadvertently disregard instructions that are vital to their immediate surroundings (Kwok & Chan, 2012). Moreover, the absence of a trusted, official digital channel can lead to the spread of misinformation and hoaxes, which further sabotages preparedness (Ameen et al., 2023). The ripple effect of these systemic failures is a heightened state of panic, disorganized movement during evacuations, and a general lack of clear guidance, all of which compromise the safety of the workforce. Without an automated and responsive framework, an organization risks not only its operational stability but also the physical security and mental well-being of its people.

To resolve these human-centered coordination issues, this study proposes the design and development of a Web-based Automated Call Tree System. This project seeks to utilize the immediate reach of SMS and email technology to establish a reliable and scalable lifeline for every employee, regardless of their physical location (Alberts &

Hayes, 2008). By leveraging cloud-based services and real-time dashboards, similar systems have shown significant improvements in alert latency and situational awareness (TechSci Research, 2024). The project strategy centers on the integration of employee data to facilitate personalized messaging that guides individuals through the emergency using a modern tech stack involving Django, Python, and Google Cloud Platform. By automating the traditional call tree, the system enables safety officers to focus on high-priority rescue tasks, secure in the knowledge that a robust, two-way communication channel is actively managing the safety of the entire DSO Team.

Objectives of the Study

General Objective

To develop a Web-based Call Tree System.

Specific Objectives

- 1) **Design** a system framework featuring automated emergency broadcasting and real-time response tracking.
- 2) **Develop** a platform using Python and the Django framework, utilizing Google Cloud Platform infrastructure and the M360 API for two-way SMS communication.
- 3) **Test** the operational functionality and notification broadcast delivery times of the system.
- 4) **Evaluate** the user and technical acceptability of the developed Web-based Automated Call Tree System based on ISO 90210.

Scope and Limitations of the Study

This study focused on the design and development of a Web-based Automated Call Tree System to optimize the earthquake preparedness and personnel tracking of the Digital Service Operations (DSO) Team. The scope encompassed the creation of a centralized administrative portal for drill management and an automated broadcast engine for location-aware notifications, the operational logic of which was illustrated in the flow diagram in **Figure 3**. The system architecture was built using Python and the Django framework, deployed on Google Cloud Platform (GCP) utilizing Cloud Run, Cloud Firestore, and Cloud Pub/Sub, and integrated with the M360 API for two-way SMS communication. These major components and their structural relationships were presented in the architectural diagram in **Figure 2**. The primary beneficiaries consisted of the Digital Service Operations (DSO) personnel.

The study was subject to specific technical and organizational boundaries. First, system operational reliability depended entirely on external cellular networks and internet service providers; therefore, message delivery was not guaranteed during a total telecommunications infrastructure collapse. Second, the target user group was limited to a selected segment of IT professionals within the Digital Service Operations (DSO) Team, excluding auxiliary departments and external service partners. Finally, the project did not include the development of physical seismic sensor hardware or building structural integrity assessments, focusing strictly on the digital communication and coordination layer.

Chapter 2

CONCEPTUAL FRAMEWORK

This chapter presents the articles, journals, and studies used to reinforce the conceptualization and framework of the research. It provides the foundational theories and processes that support the study's development. Furthermore, it details the necessary information for selecting materials and components used in the prototype's construction. Finally, this chapter discusses the conceptual model utilized in the research and provides operational definitions for the terms used throughout the study.

Review of Related Literature and Studies

Philippines maintains a high-risk profile due to its geographical location within the Pacific Ring of Fire, a reality that necessitates proactive disaster risk reduction policies. Punzalan and Mendoza (2018) argue that the cornerstone of any resilient community is its communication infrastructure. When communication links are broken or delayed, the psychological and physical vulnerability of individuals increases. Consequently, resilience is defined not only by the structural integrity of buildings but by the speed and accuracy of information disseminated to individuals within them during a crisis.

Traditional emergency protocols, such as manual phone trees, fail to meet the

operational demands of modern, distributed work environments. Cutter and Emrich (2016) highlight that human stress and fatigue during an actual earthquake lead to systemic failures in manual call workflows. Research indicates that manual tracking and headcount procedures are notoriously slow and remain prone to human errors that misdirect emergency responders (Lim & Rodrigues, 2018). Localized randomized trials in the Philippines reveal that manual recall methods suffer from high variability in response times compared to automated text-based solutions, which provide a more uniform and rapid dissemination of safety instructions (Homier et al., 2018).

The shift toward digitized disaster management allows for scalable and context-aware emergency responses. By utilizing near real-time global seismic feeds like the USGS Earthquake Notification Service (ENS), automated systems trigger alerts based on precise magnitude and location data before manual reports can be filed (USGS, 2024). Automated call trees directly address the limitations of human operators who easily become overwhelmed during a genuine disaster (Lau, Kwok, & Chan, 2012). By leveraging cloud-based services and real-time dashboards, organizations significantly reduce alert latency and improve situational awareness among all stakeholders (Tan et al., 2017). The use of multi-channel communication, such as SMS and email via gateway APIs, ensures that critical safety directives reach employees regardless of their physical location (Alberts & Hayes, 2008). In the Philippine context, Estuar (2015) notes that integrating personnel data allows for personalized messaging that fosters higher engagement and more efficient coordination during high-stress drills.

The reviewed literature emphasizes that while manual notification systems are familiar, they are inherently flawed due to human-induced latency and the high-stress nature of seismic events. In terms of purpose, these closely related studies seek to optimize disaster risk reduction and community resilience. Methodologically, the researchers utilize randomized trials, comparative latency analyses, and infrastructure scaling assessments to measure response efficiency. The principal findings consistently demonstrate that manual tracking and traditional phone trees introduce severe bottlenecks and high response variability, whereas automated, multi-channel protocols significantly reduce alert latency. Consequently, the overarching conclusion across the literature is that automation represents the most viable path toward achieving zero-delay communication. While existing studies focus on general emergency alerts, there is a clear gap regarding localized, web-based systems that specifically integrate global monitoring feeds like the USGS Notification Service with local SMS gateways (M360) for earthquake-specific drills in the Philippines. This study addresses this gap by developing a system that combines near real-time data ingestion from global sources with real-time data persistence (Firestore) and serverless hosting (Cloud Run) to ensure the framework remains operational even when local physical infrastructure is compromised.

Conceptual Model of the Study

The conceptual framework of this research utilizes the Input-Process-Output (IPO) model to establish the developmental framework of the prototype. Figure 1 illustrates the structural relationship between the foundational requirements, the methodology stages, and the expected deliverables.

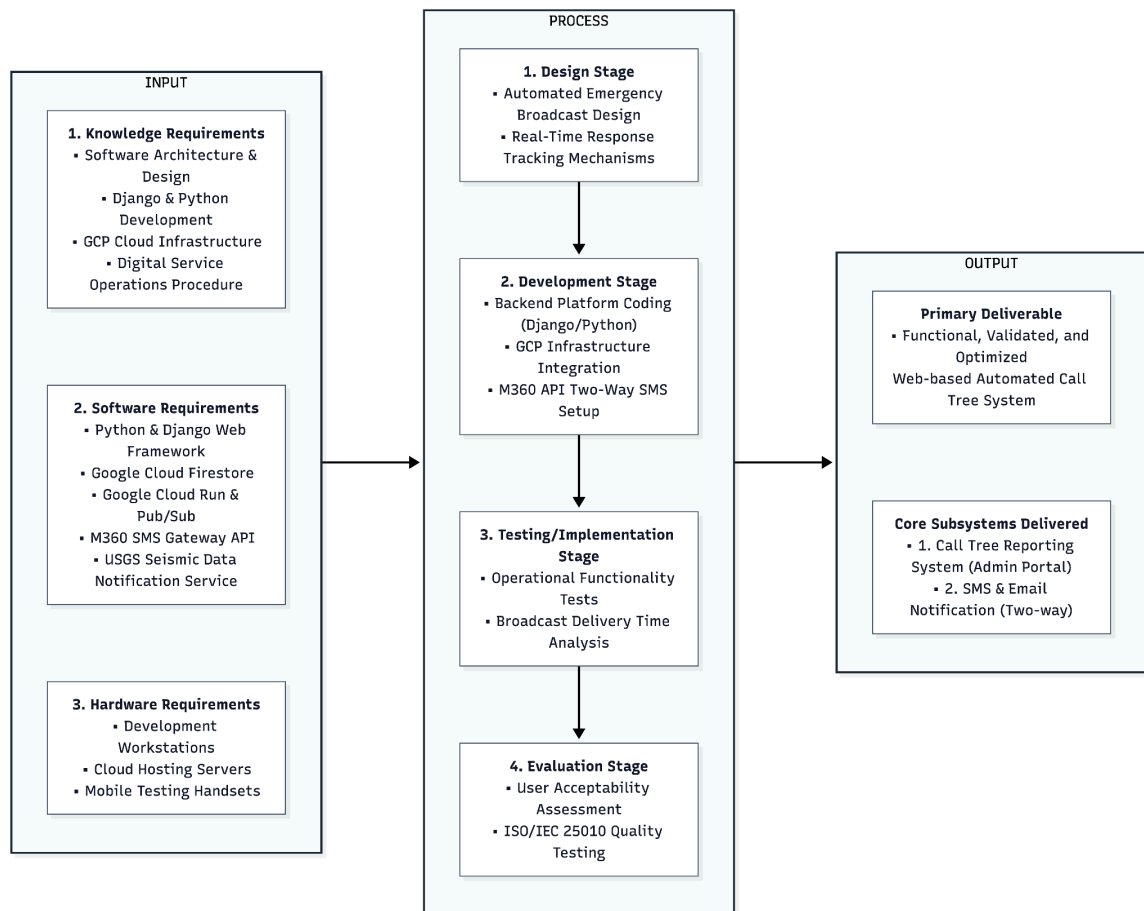


Figure 1. Conceptual Model of the Web-based Automated Call Tree System

The Input block consists of the explicit technical and operational resources required to initiate the project. The knowledge requirements encompass software engineering architecture, full-stack Python and Django development methodologies,

serverless Google Cloud Platform (GCP) configurations, and the specific guidelines contained within the existing Digital Service Operations Procedure. This organizational procedure provides the operational parameters and logic necessary to structure the emergency communication workflows. The software requirements comprise the actual programming languages, developer frameworks, cloud assets, and external data interfaces used to construct the system. These include the Python language, the Django framework, Google Cloud Firestore for real-time document storage, Google Cloud Run for serverless processing, and Google Cloud Pub/Sub for asynchronous routing. Crucially, this block incorporates the M360 SMS Gateway API to bridge cell networks and the USGS Seismic Data Notification Service to supply live external earthquake feeds. The hardware requirements identify the physical infrastructure, which includes local engineering workstations, GCP cloud hosting clusters, and mobile testing handsets used to verify cellular transmission.

The Process block outlines the systematic engineering lifecycle executed across four sequential phases: Design, Development, Implementation, and Evaluation. The Design Stage centers on mapping the structural architecture of the platform, detailing the database schemas, and outlining the workflows for automated emergency broadcasting, real-time employee response tracking, and automated Emergency Management Team (EMT) escalation pathways. The Development Stage focuses on translating these designs into functional components by executing backend platform coding in Python and Django, configuring the GCP cloud infrastructure environment, and setting up the two-way M360 API communications layer. The Testing and Implementation Stage deploys the system to an active state to conduct thorough operational functionality assessments and execute

latency tests that analyze the specific notification broadcast delivery times. The Evaluation Stage concludes the workflow by conducting user acceptability assessments and gathering data to measure the overall technical quality of the software platform against the metrics defined by the ISO/IEC 25010 standards.

The Output block represents the direct, tangible deliverable achieved once the input components successfully undergo the process phases. This results in a functional, validated, and optimized Web-based Automated Call Tree System dedicated to the Digital Service Operations Team. The final output features two core sub-systems: a centralized Call Tree Reporting System that offers an interactive dashboard for administrative overview, and automated SMS and Email Notification delivery mechanisms that establish the primary communication loop. Together, these outputs eliminate manual tracking bottlenecks and secure instantaneous, verifiable personnel accountability during seismic emergencies.

Operational Definition of Terms

The following terms are defined according to how they are used within the scope of this study:

Automated Call Tree System refers to the web-based prototype developed in this study that utilizes pre-defined logic to disseminate safety alerts via SMS and email without manual intervention.

Cloud Run refers to the serverless Google Cloud Platform (GCP) service used to host the system, ensuring that the application remains accessible to all users simultaneously during high-traffic earthquake drills.

DSO Team is the Digital Service Operations Team, which serves as the primary group responsible for managing emergency communication workflows and is the intended beneficiary of the Web-based Automated Call Tree System.

Earthquake Drill refers to the simulated emergency scenarios used to evaluate the speed, reliability, and user stress reduction provided by the system.

Google Firestore refers to the real-time NoSQL database used within the prototype to store and synchronize personnel safety status updates and contact information instantaneously.

M360 refers to the multi-channel SMS gateway API used to facilitate two-way communication between the system administrators and the team members within the

Philippine telecommunications network.

Pub/Sub refers to the asynchronous messaging service within the tech stack that manages the high-volume queuing of notifications to ensure every employee is reached sequentially and reliably.

Python and Django refer to the core programming language and web framework, respectively, used to build the logic and administrative interface of the safety dashboard.

USGS Earthquake Notification Service (ENS) refers to the external data source used to provide real-time earthquake parameters (magnitude and location) that trigger the system's automated call tree logic.

Chapter 3

METHODOLOGY

This chapter presents the isometric drawings, block diagrams, and process flow diagrams related to the implementation of the study. It provides a detailed discussion of the technical design and the step-by-step procedures undertaken during the prototype's fabrication. Furthermore, the operational, testing, and evaluation procedures are discussed to establish the qualifications by which the project's performance is measured.

Project Design

The design of the Web-based Automated Call Tree System is grounded in a cloud-native architectural framework, specifically engineered to eliminate the manual latencies inherent in traditional emergency call trees. The system functions as a highly responsive intermediary that bridges global seismic data with local organizational safety protocols.

System Modules and Functional Components. The architecture is divided into four primary functional modules that operate in a synchronized, event-driven manner:

1. **USGS Data Ingestion Module:** This module serves as the entry point for seismic data. It is configured to listen for incoming webhooks or email alerts from the **USGS Earthquake Notification Service (ENS)**. The module extracts critical metadata, including the magnitude, timestamp, and coordinates

(Latitude/Longitude) of the seismic event.

2. **Proximity Analysis & Logic Engine:** Built using the **Django Web Framework**, this engine performs a spatial comparison between the epicenter reported by USGS and the geofenced locations of the DSO Operations Team stored in the database. It applies a conditional "Trigger Filter"—if the magnitude is above the set threshold (e.g., 5.0) and the distance is within the risk radius, the call tree is activated.
3. **Cloud Dissemination Module:** This component leverages **Google Cloud Pub/Sub** for asynchronous messaging. By decoupling the notification logic from the web server, the system ensures that thousands of alerts can be queued and sent simultaneously via the **M360 SMS API** and **SMTP Email servers** without crashing the main application.
4. **Accountability & Feedback Module:** This provides the two-way communication loop. It captures inbound SMS replies from employees and updates the **Google Firestore NoSQL database** in real-time. This data is then rendered on the Administrative Dashboard to provide an instant headcount.

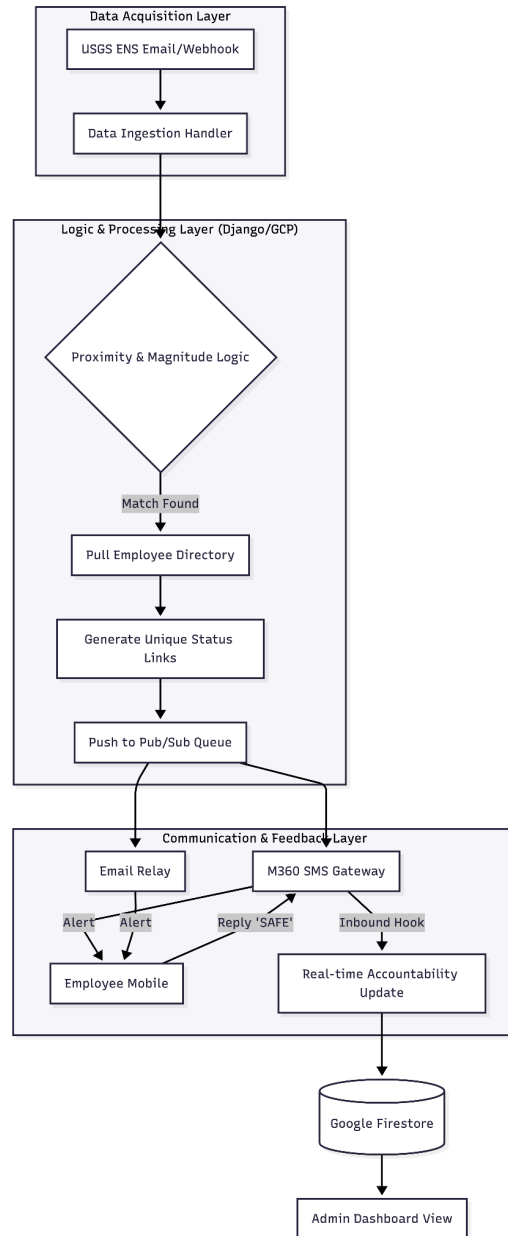


Figure 2. Architectural Diagram of the Call Tree System

The structural design represented in Figure 2 served as the primary blueprint for configuring the cloud services and database models. By establishing distinct acquisition, logic, and feedback layers, the researcher was able to build and deploy each module independently on Google Cloud, thereby ensuring that high-volume message traffic did

not compromise the performance of the administrative database during simulated disaster scenarios.

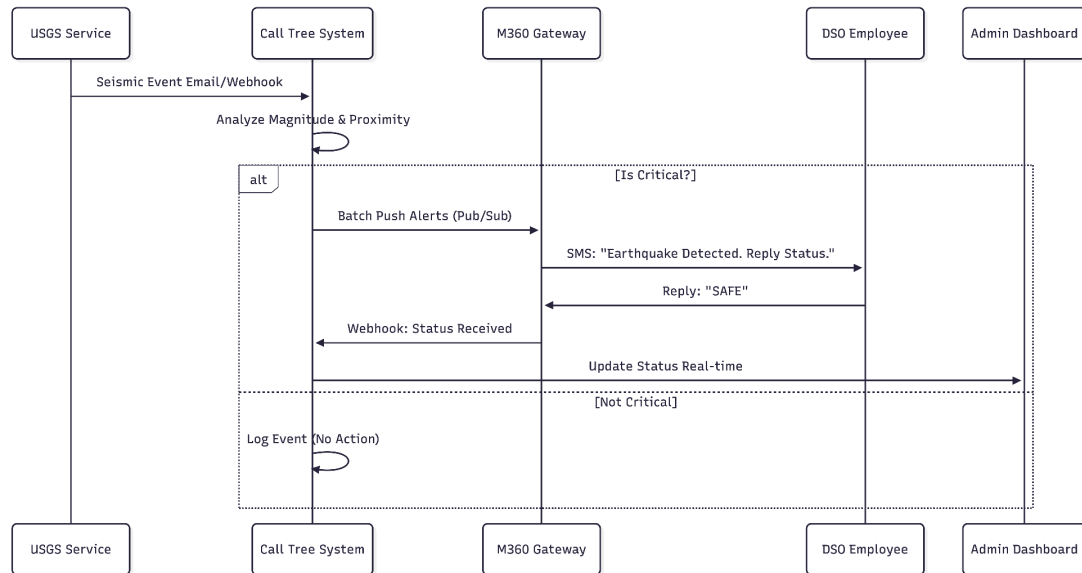


Figure 3. Operational Flow of a Seismic Event Trigger

The dynamic interaction illustrated in Figure 5 guided the behavior of the backend endpoints. This sequence model was used to program the asynchronous message delivery queues, defining how external webhooks from USGS and response payloads from the M360 gateway mapped to real-time dashboard UI updates. It guaranteed that data was preserved and synchronized sequentially.

Project Development

The development phase utilized the Agile Software Development Life Cycle (SDLC) framework, which was characterized by iterative coding, constant testing, and incremental refinement. This approach ensured that technical bottlenecks were identified early in the implementation of the cloud infrastructure.

Procedural Steps in Development. The project execution spanned several key phases:

1. **Requirement Analysis and Planning:** The researcher conducted an audit of the DSO Operations Team's existing emergency workflows to determine target response windows and baseline communication latencies.
2. **Environment Setup and Database Modeling:** The local development space was initialized using Python 3.x, and a scalable NoSQL database schema was constructed within Google Firestore to handle rapid, concurrent write operations.
3. **Backend Integration:** The central "Call Tree Logic" was coded in Django. This step involved writing ingestion functions to securely parse incoming USGS email alerts.
4. **Frontend Dashboard Fabrication:** A responsive administrative portal was constructed using Tailwind CSS, establishing a clean interface to track drill progress.
5. **SMS Gateway Integration:** The M360 API was connected using asynchronous request handlers, allowing the system to send bulk messages without freezing the user interface.

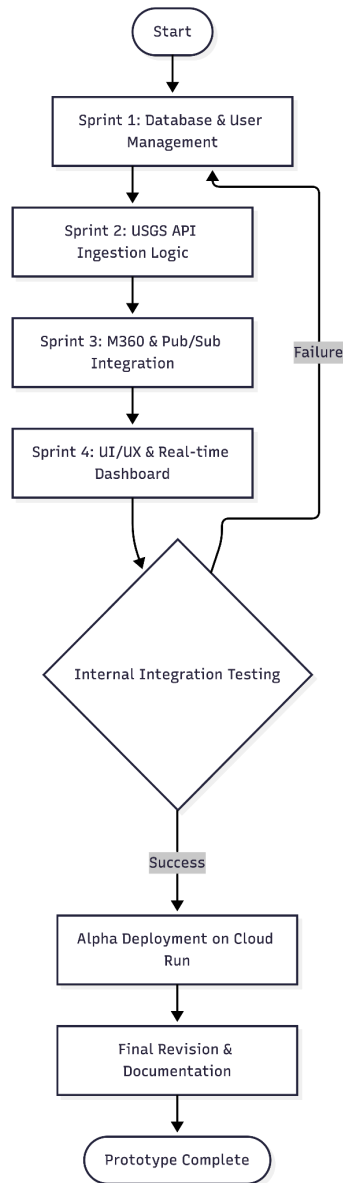


Figure 4. Developmental Progression

The flowchart in Figure 4 mapped out the developmental progression of the prototype. This iterative approach enabled the structured debugging of each system component. If integration issues occurred during testing—particularly when bridging Cloud Pub/Sub with the M360 gateway—the project reverted to the corresponding sprint, ensuring all architectural errors were solved prior to cloud deployment.

System Operational and Compliance Workflow

To guarantee disaster preparedness compliance under corporate and national guidelines, the operational workflow was bifurcated between the System Administrator and the DSO Employee. This dual-path design ensured total acknowledgment and reporting compliance.

Administrator Operational Flow. The Safety Officer followed a controlled, manual execution path:

1. **Configuration:** The Administrator pre-loaded employee directories, geofences, and compliance windows.
2. **System Health Check:** Prior to launching active drills, the Admin monitored API balances and cloud container statuses.
3. **Active Monitoring:** Upon manual initiation, the Safety Officer supervised response analytics on the dashboard.
4. **Non-Compliance Intervention:** The dashboard automatically flagged non-responsive employees, allowing the Admin to initiate targeted re-broadcasting and escalation steps.
5. **Analytical Audit:** After drill completion, compliance reports detailing response latencies were exported.

DSO Employee Compliance Flow. The employee workflow integrated physical drills with digital tracking:

1. **Alert Reception:** The employee received urgent multi-channel instructions

customized to their work location.

2. **Physical Safety Execution:** The employee completed physical maneuvers (such as "Drop, Cover, and Hold") and moved to safety.
3. **Safety Status Reporting:** Once secure, the employee completed the loop by replying "SAFE" or "HELP" via SMS, or by tapping a unique web status link.
4. **Escalation Avoidance:** Successful acknowledgment logged the employee's status and halted the automated reminder loops.

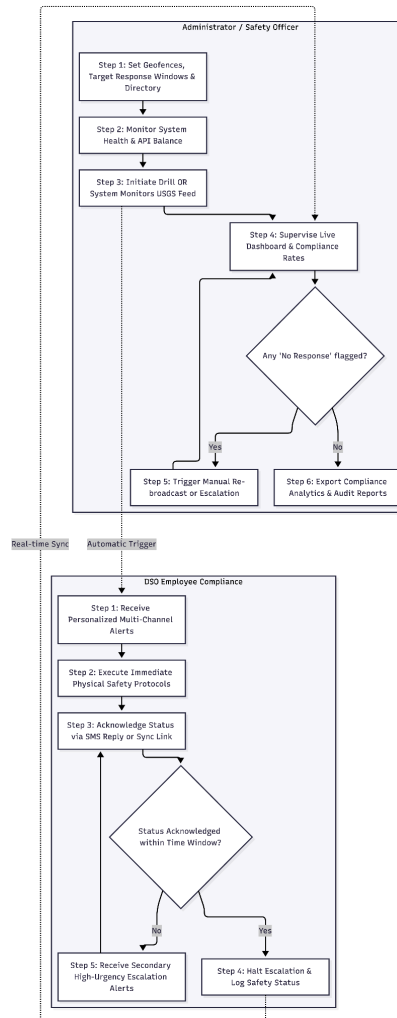


Figure 5. Developmental Progression

The flowchart in Figure 7 visually coordinated the interaction between administrative oversight and end-user safety response workflows. This diagram served as a structural guide during system coding, ensuring that database updates from the employee's mobile responses synchronized seamlessly with the live administrative view.

Operation and Testing Procedure

The operational effectiveness of the system is validated through a series of structured testing procedures, ensuring that the transition from a "Data Trigger" to an "Employee Alert" occurs within acceptable latency parameters.

Operational Procedure

1. **Administrator Authentication:** The Safety Officer signs in securely using the admin portal credentials.
2. **Drill Parameters Configuration:** The Administrator inputs or verifies the incident details (such as simulating a USGS earthquake alert magnitude or defining the targeted employee geofences) on the settings page.
3. **Manual Trigger Initiation:** The Administrator clicks the "**Initiate Call Tree Alert**" button, triggering the backend script.
4. **Interactive Dashboard Monitoring:** The Administrator monitors the status table and charts on the screen as employees reply to their SMS and email notifications in real-time.

Testing Procedure

- **Admin Trigger Test:** Confirming that clicking the dashboard trigger button successfully converts database records into Pub/Sub message payloads.
- **Two-Way Status Update Test:** Ensuring that replying SAFE or HELP from a test mobile device correctly triggers the M360 webhook and instantly updates the admin monitor.
- **Delivery Latency Measurement:** Logging the exact timestamps from the Admin trigger action to the physical SMS arrival on user devices to determine overall transit speed.

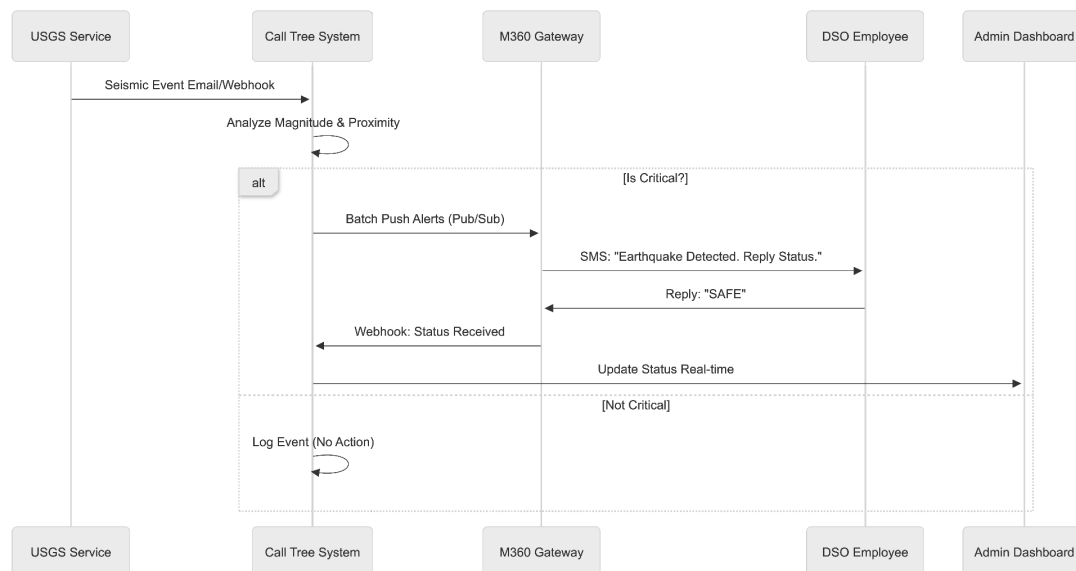


Figure 6. Operational Flow of a Seismic Event Trigger

To guarantee disaster preparedness compliance under corporate and national standards (e.g., National Disaster Risk Reduction and Management Council guidelines in the Philippines), the system divides its operational workflows between two primary

actors: the **System Administrator (Safety Officer)** and the **DSO Employee**. This dual-path process ensures systematic monitoring and maximum acknowledgment compliance.

Administrator Operational Flow

1. **Directory and Parameter Configuration:** The Administrator pre-loads employee profiles, communication routes (primary SMS, backup email), work location geofences, and target compliance response windows (e.g., a target of 100% response within 5 minutes).
2. **System Health Verification:** Prior to active drills or ongoing automated monitoring, the Administrator audits API balances (M360 SMS credits) and monitors GCP container statuses via the Django administrative terminal.
3. **Drill/Incident Monitoring:** Upon system trigger (either manually forced for scheduled drills, or automatically activated by USGS incoming notifications), the Admin supervises the live dashboard.
4. **Active Intervention (Non-Compliance Follow-up):** As responses stream in, the system flags non-compliant employees (those failing to respond within the target threshold). The Admin initiates localized, targeted re-broadcasting rules or manual escalation procedures.
5. **Post-Event Compliance Reporting:** The Administrator exports detailed analytics showing historical response times, systemic message latency, and individual employee compliance summaries for management audit.

Administrator Operational Flow

1. **Directory and Parameter Configuration:** The Administrator pre-loads employee profiles, communication routes (primary SMS, backup email), work location geofences, and target compliance response windows (e.g., a target of 100% response within 5 minutes).
2. **System Health Verification:** Prior to active drills or ongoing automated monitoring, the Administrator audits API balances (M360 SMS credits) and monitors GCP container statuses via the Django administrative terminal.
3. **Drill/Incident Monitoring:** Upon system trigger (either manually forced for scheduled drills, or automatically activated by USGS incoming notifications), the Admin supervises the live dashboard.
4. **Active Intervention (Non-Compliance Follow-up):** As responses stream in, the system flags non-compliant employees (those failing to respond within the target threshold). The Admin initiates localized, targeted re-broadcasting rules or manual escalation procedures.
5. **Post-Event Compliance Reporting:** The Administrator exports detailed analytics showing historical response times, systemic message latency, and individual employee compliance summaries for management audit.

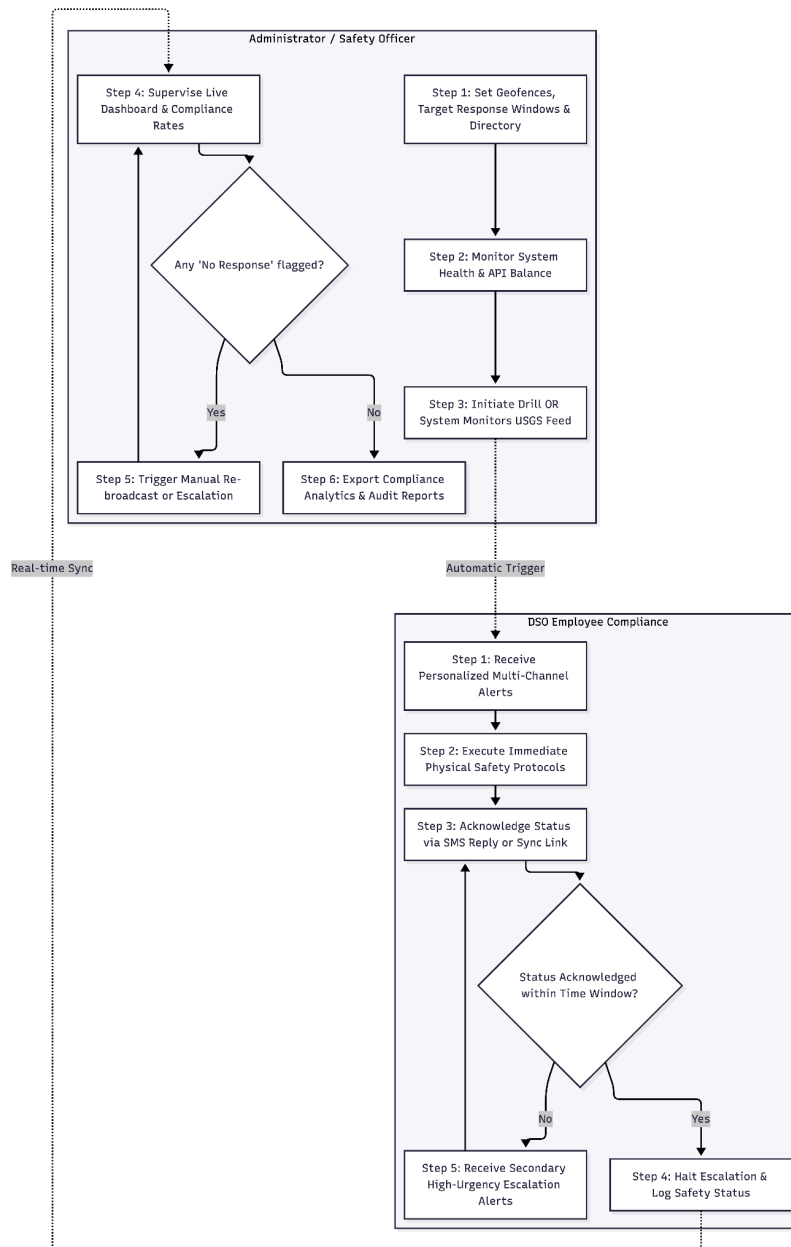


Figure 7. Administrator and Employee Compliance Operational Flowchart

Evaluation Procedure

To evaluate the technical quality and user acceptability of the Web-based Automated Call Tree System, a structured evaluation will be conducted using an assessment instrument aligned with the ISO/IEC 25010 Quality Model. The evaluation cohort consists of ten (10) representative personnel from the Digital Service Operations (DSO) Team (End-Users and Administrative Safety Officers).

Step-by-Step Evaluation Protocol:

1. Pre-Evaluation Orientation:

- Evaluators are briefed on the objectives of the study, the scope of the Web-based tool, and the specific ISO evaluation criteria (Functional Suitability, Performance Efficiency, and Usability).
- Each evaluator receives an evaluation packet containing the standardized survey questionnaire and scenario task sheets.

2. Scenario Execution & Test Demonstration:

- Step 2a (Admin Flow): Designated safety officer evaluators log into the Administrative Dashboard, configure drill parameters (target response windows), and initiate a simulated seismic broadcast trigger.
- Step 2b (User Response Flow): End-user evaluators receive simulated automated

SMS and email alerts on mobile testing handsets, interact with the notification payload (replying "SAFE" or tapping the unique web status link), and observe status updates.

- Step 2c (Dashboard Real-time Audit): Evaluators verify that inbound mobile responses synchronize instantaneously on the live administrative dashboard view.

Numerical Scale	Descriptive Rating
5.0	Highly Acceptable
4.0	Very Acceptable
3.0	Acceptable
2.0	Fairly Acceptable
1.0	Not Acceptable